



E. Fermi - R. Guttuso
Istituto Statale Istruzione Superiore
Via N. Maccarone, 4 - Giarre (CT) - 095613055



BLACK RADIATORS

RESCUE SIMULATION

TEAM

I.I.S. «E. Fermi - R. Guttuso» · Giarre (CT), Italy | League: Rescue Simulation

- ◆ **Marino Marco** — Team Captain & Software Developer. Exploration and path-planning algorithms (wall-following + BFS to the frontier) and map management. *National finalist at OliCyber.IT.*
- ◆ **Pennino Cristian Francesco** — Software Developer. Computer-vision system and integration of the YOLO model for victim detection and classification. *National finalist at OliCyber.IT and national 3rd-place winner at the STMMicroelectronics STM32 contests.*
- ◆ **Arcidiacono Gabriele** — Designer & Software Developer. Robot design and development of the exploration and navigation logic.
- ◆ **Puglisi Paolo Michael** — Hardware Lead. Robot design and construction, sensor selection and creation of the test environments.

Results & competitions: RoboCupJunior Rescue Simulation 2026 — **1st place nationally** and qualification for the **World Championship** (Incheon, July 2026). Members are multiple national-competition awardees: OliCyber.IT, STMMicroelectronics STM32, Italian Informatics Olympiad.



The team at national competitions



RoboCupJunior 2026 — National Finals, I.T. Archimede (Catania)



Italian Cybersecurity Olympiad 2026 — National Competition

ROBOT & HARDWARE



Selection strategy. A minimal sensor set to stay within the **3000** budget: the **LIDAR** replaces many distance sensors (multiple samples + depth for 3D letters); **two 64x64 side cameras** cover both walls of a corridor instead of a single front camera; the **colour sensor** recognises holes/checkpoints/swamps without image processing.

- ◆ **LIDAR** — measures front and side distance with multiple samples for precise navigation; depth variance identifies the **fake 3D letters**.
- ◆ **Inertial Unit (IMU)** — provides the orientation (yaw) to turn toward the cardinal directions and correct the heading in real time.
- ◆ **2x Camera (64x64)** — recognise victims by combining **YOLO** (symbols on a white background) and **HSV** analysis (coloured cognitive targets).
- ◆ **Colour Sensor** — detects black holes by comparing the RGB sum with a fixed threshold (**BLACK_THRESHOLD**), plus checkpoints, swamps and coloured passages.
- ◆ **GPS** — absolute position converted into grid coordinates to build the map and plan the paths.
- ◆ **Wheel Encoder** — measures travelled distance for precise short forward/reverse manoeuvres via odometry.
- ◆ **Emitter / Receiver** — communication with the supervisor: sending victim coordinates, map and exit; receiving score and game time.

SOFTWARE

Python & Weboots OpenCV NumPy Ultralytics YOLO Matplotlib

- ◆ **Exploration** — *right-wall following* with proportional control on a **1 cm occupancy grid** inflated into **configuration-space** (3 cm padding around walls and holes).
- ◆ **Planning** — when the local area is exhausted, a **padding-tolerant BFS** to the nearest frontier; the path is smoothed with a **line-of-sight (Bresenham)** test into a few fast straight legs.
- ◆ **Robustness** — a GPS-displacement anti-stuck **watchdog** and recovery from holes/swamps projected onto the map.
- ◆ **Mapping** — dual representation: a 1 cm C-space grid for planning + a **3 cm matrix** in Erebus format submitted for the *mapping bonus*.
- ◆ **Victim detection** — hybrid pipeline: **YOLO** for the letters ($\Phi/\Psi/\Omega \rightarrow H/S/U$) + a **scanline** that sums the 5 rings of the cognitive targets ($\rightarrow F/P/C/O$), rejecting the fake 3D letters.

```
# Padding-tolerant BFS: find the nearest frontier
def bfs_frontier(grid, start):
    queue = deque([(start, 0)]) # (cell, padding steps)
    seen = {start: 0}
    while queue:
        cell, pad = queue.popleft()
        if grid[cell] == UNKNOWN: # frontier found
            return reconstruct(cell)
        for nb in neighbors(cell):
            if grid[nb] == WALL: continue
            p = pad + 1 if grid[nb] == PADDING else 0
            if p <= MAX_PAD and better(nb, p, seen):
                seen[nb] = p; queue.append((nb, p))
```

Flowchart — main loop

